

Guía Completa de SymmetricDS

Sincronización de Datos entre PostgreSQL

Tabla de Contenidos

1. [¿Qué es SymmetricDS?](#)
 2. [Arquitectura y Conceptos](#)
 3. [Configuración Inicial](#)
 4. [Sincronización Unidireccional](#)
 5. [Sincronización Bidireccional](#)
 6. [Verificación y Troubleshooting](#)
 7. [Casos de Uso Comunes](#)
-

¿Qué es SymmetricDS?

SymmetricDS es una herramienta de replicación de datos en tiempo real entre bases de datos. Permite sincronizar tablas entre múltiples nodos de forma automática, soportando arquitecturas maestro-esclavo, multi-maestro, hub-and-spoke, y más.

Características principales:

-  Replicación en tiempo real
 -  Soporte para múltiples bases de datos (PostgreSQL, MySQL, Oracle, SQL Server, etc.)
 -  Sincronización bidireccional y unidireccional
 -  Manejo de conflictos
 -  Carga inicial automática
 -  Reintentos automáticos en caso de fallas
-

Arquitectura y Conceptos

Conceptos Fundamentales

1. Node Group (Grupo de Nodos)

Agrupar nodos que tienen el mismo rol en la arquitectura.

Tabla: sym_node_group

```
sql

-- Ejemplo: Crear grupos de nodos
INSERT INTO sym_node_group (node_group_id, description)
VALUES ('central', 'Nodo Central - Matriz');

INSERT INTO sym_node_group (node_group_id, description)
VALUES ('sucursal', 'Nodos de Sucursales');
```

¿Para qué sirve?

- Define roles: quién es central, quién es sucursal
- Permite crear reglas de sincronización por grupos completos
- Facilita agregar nuevos nodos sin reconfigurar todo

Ejemplos comunes:

- master / slave
- central / sucursal
- headquarters / branch
- server / client

2. Node Group Link (Enlace entre Grupos)

Define la relación y dirección de sincronización entre grupos de nodos.

Tabla: sym_node_group_link

```
sql
```

```
-- Define que 'central' puede enviar datos a 'sucursal'
INSERT INTO sym_node_group_link (
    source_node_group_id,
    target_node_group_id,
    data_event_action
) VALUES (
    'central',
    'sucursal',
    'W' -- 'W' = Wait (espera), 'P' = Push (empuja inmediatamente)
);
```

¿Para qué sirve?

- Establece la topología de red: quién puede enviar datos a quién
- Es un **requisito previo** para crear routers
- Define el tipo de sincronización (push vs pull)

Valores de `data_event_action`:

- `(W)` (Wait): Los cambios esperan a ser solicitados (pull)
- `(P)` (Push): Los cambios se envían inmediatamente

3. Channel (Canal)

Agrupar tablas relacionadas y controlar cómo se sincronizan.

Tabla: `sym_channel`

```
sql

INSERT INTO sym_channel (
    channel_id,
    processing_order,
    max_batch_size,
    enabled
) VALUES (
    'default',
    1,      -- Orden de procesamiento (menor = primero)
    1000,   -- Máximo de cambios por lote
    1       -- Habilitado
);
```

¿Para qué sirve?

- Prioriza qué tablas se sincronizan primero
- Controla el tamaño de los lotes de sincronización
- Permite pausar/reanudar sincronización por grupos de tablas

Canales comunes:

- `default`: Para la mayoría de tablas
- `config`: Para tablas de configuración (alta prioridad)
- `transaction`: Para transacciones críticas
- `bulk`: Para datos masivos (baja prioridad)

Ejemplo con múltiples canales:

```
sql

-- Canal de alta prioridad para configuración
INSERT INTO sym_channel (channel_id, processing_order, max_batch_size, enabled)
VALUES ('config', 1, 100, 1);

-- Canal normal para datos transaccionales
INSERT INTO sym_channel (channel_id, processing_order, max_batch_size, enabled)
VALUES ('transaction', 5, 1000, 1);

-- Canal de baja prioridad para datos históricos
INSERT INTO sym_channel (channel_id, processing_order, max_batch_size, enabled)
VALUES ('history', 10, 5000, 1);
```

4. Trigger (Disparador)

Detecta cambios (INSERT, UPDATE, DELETE) en una tabla específica.

Tabla: `sym_trigger`

```
sql
```

```

INSERT INTO sym_trigger (
  trigger_id,      -- Identificador único
  source_table_name, -- Tabla a monitorear
  channel_id,      -- Canal al que pertenece
  sync_on_insert,  -- ¿Sincronizar INSERTs? (1=si, 0=no)
  sync_on_update,  -- ¿Sincronizar UPDATES?
  sync_on_delete,  -- ¿Sincronizar DELETES?
  last_update_time,
  create_time
) VALUES (
  'trg_clientes',
  'clientes',    -- Nombre exacto de la tabla en la BD
  'default',     -- Usa el canal 'default'
  1, 1, 1,      -- Sincroniza INSERT, UPDATE y DELETE
  NOW(),
  NOW()
);

```

¿Para qué sirve?

- SymmetricDS crea triggers de base de datos automáticamente
- Captura cada cambio que ocurre en la tabla
- Guarda los cambios en tablas internas para sincronización

Opciones avanzadas (no las usamos, pero existen):

- `sync_on_insert_condition`: Condición SQL para filtrar INSERTs
- `sync_on_update_condition`: Solo sincronizar si se cumplen condiciones
- `excluded_column_names`: Columnas que NO se sincronizan
- `sync_on_incoming_batch`: ¿Sincronizar cambios que vienen de otros nodos?

Ejemplo con filtros:

```
sql
```

```
-- Solo sincronizar clientes activos
```

```
INSERT INTO sym_trigger (  
    trigger_id,  
    source_table_name,  
    channel_id,  
    sync_on_insert,  
    sync_on_update,  
    sync_on_delete,  
    sync_on_insert_condition,  
    sync_on_update_condition,  
    last_update_time,  
    create_time  
) VALUES (  
    'trg_clientes_activos',  
    'clientes',  
    'default',  
    1, 1, 1,  
    'estado = "ACTIVO"', -- Solo INSERT de clientes activos  
    'estado = "ACTIVO"', -- Solo UPDATE de clientes activos  
    NOW(),  
    NOW()  
);
```

5. Router (Enrutador)

Define **QUIÉN** recibe los cambios capturados por los triggers.

Tabla: `sym_router`

```
sql
```

```

INSERT INTO sym_router (
  router_id,          -- Identificador único
  source_node_group_id, -- Grupo que ENVÍA datos
  target_node_group_id, -- Grupo que RECIBE datos
  router_type,        -- Tipo de enrutamiento
  create_time,
  last_update_time
) VALUES (
  'rt_central_to_sucursal',
  'central',          -- Origen: nodos centrales
  'sucursal',         -- Destino: nodos sucursales
  'default',          -- Envía a TODOS los nodos del grupo destino
  NOW(),
  NOW()
);

```

¿Para qué sirve?

- Define a qué nodos destino van los datos
- Permite filtrar datos por nodo específico
- Crea las rutas de sincronización

Tipos de Router (**router_type**):

1. **default**: Envía a TODOS los nodos del grupo destino
2. **column**: Filtra por el valor de una columna
3. **bsh** (BeanShell): Usa un script para decidir a quién enviar
4. **subselect**: Usa una subconsulta SQL para filtrar

Ejemplo con router de columna:

```
sql
```

-- Cada sucursal solo recibe SUS clientes

```
INSERT INTO sym_router (  
    router_id,  
    source_node_group_id,  
    target_node_group_id,  
    router_type,  
    router_expression, -- Expresión para filtrar  
    create_time,  
    last_update_time  
) VALUES (  
    'rt_clientes_por_sucursal',  
    'central',  
    'sucursal',  
    'column',  
    'sucursal_id=:EXTERNAL_ID', -- Solo envía si sucursal_id coincide con external_id del nodo  
    NOW(),  
    NOW()  
);
```

En este ejemplo, si la tabla `clientes` tiene una columna `sucursal_id`, cada sucursal solo recibirá los clientes donde `sucursal_id` coincida con su `external_id`.

6. Trigger Router (Unión Trigger-Router)

CONECTA un trigger con un router. Es el último paso que activa la sincronización.

Tabla: `sym_trigger_router`

sql


```

INSERT INTO sym_trigger_router (
  trigger_id,      -- Qué trigger (qué tabla)
  router_id,       -- Qué router (a quién enviar)
  initial_load_order, -- Orden en carga inicial (menor = primero)
  enabled,         -- ¿Activo? (1=sí, 0=no)
  create_time,
  last_update_time
) VALUES (
  'trg_clientes',
  'rt_central_to_sucursal',
  100,           -- Se carga en orden 100
  1,             -- Habilitado
  NOW(),
  NOW()
);

```

¿Para qué sirve?

- Activa la sincronización: "cuando la tabla X cambie, envía a los nodos Y"
- Controla el orden de carga inicial (importante para llaves foráneas)
- Permite habilitar/deshabilitar sincronización sin borrar configuración

initial_load_order - ¿Por qué es importante?

Cuando una sucursal nueva se registra, recibe una "carga inicial" de TODOS los datos. El orden importa por las relaciones:

```

sql

-- Orden correcto para carga inicial:
-- 1. Tablas maestras primero

INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, enabled, ...)
VALUES ('trg_paises', 'rt_central_to_sucursal', 10, 1, ...); -- Primero

INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, enabled, ...)
VALUES ('trg_ciudades', 'rt_central_to_sucursal', 20, 1, ...); -- Segundo (depende de paises)

INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, enabled, ...)
VALUES ('trg_clientes', 'rt_central_to_sucursal', 100, 1, ...); -- Último (depende de ciudades)

```

7. Node (Nodo)

Representa una instancia específica de base de datos en la red de sincronización.

Tabla: sym_node

```
sql

INSERT INTO sym_node (
  node_id,          -- ID único del nodo (generalmente = external_id)
  node_group_id,    -- A qué grupo pertenece
  external_id,      -- ID externo (del .properties)
  sync_enabled,     -- ¿Sincronización activa?
  sync_url,         -- URL donde otros se conectan a ESTE nodo
  created_at_node_id -- Qué nodo lo creó
) VALUES (
  '000',
  'central',
  '000',
  1,
  'http://symmetricds:31415/sync/quito',
  '000' -- Se creó a sí mismo
);
```

¿Para qué sirve?

- Registra cada nodo en la red
- Almacena información de conectividad
- Rastrea el estado de sincronización

8. Node Security (Seguridad del Nodo)

Controla la autenticación y permisos de registro de cada nodo.

Tabla: sym_node_security

```
sql
```

```

INSERT INTO sym_node_security (
  node_id,
  node_password,      -- Contraseña para autenticación
  registration_enabled, -- ¿Puede registrarse? (1=sí, 0=no)
  registration_time,   -- Cuando se registró (NULL si aún no)
  initial_load_enabled, -- ¿Recibirá carga inicial?
  created_at_node_id
) VALUES (
  '002',
  'guayaquil123',
  1,          -- ☒ Registro habilitado
  NULL,      -- Se llenará automáticamente al registrarse
  1,          -- ☒ Carga inicial habilitada
  '000'
);

```

¿Para qué sirve?

- Seguridad: solo nodos autorizados pueden registrarse
- Control de carga inicial: decides si un nodo recibe todos los datos históricos
- Auditoría: registra cuándo se conectó cada nodo

Configuración Inicial

Estructura de Archivos .properties

Los archivos `.properties` configuran cada instancia de SymmetricDS.

Nodo Central (Quito)

Archivo: `quito.properties`

properties

Nombre único de este motor SymmetricDS

engine.name = quito

Configuración de base de datos

db.driver = org.postgresql.Driver

db.url = jdbc:postgresql://postgres-quito:5432/quito_db

db.user = postgres

db.password = postgres

Identificación del nodo

group.id = central # A qué grupo pertenece

external.id = 000 # ID único de este nodo

URLs de sincronización

registration.url = # Vacío porque ES el central (no se registra en nadie)

sync.url = http://symmetricds:31415/sync/quito # Donde otros se conectan a ÉL

Comportamiento

auto.registration = false # El central no se auto-registra

initial.load.create.first = true # Crea estructuras si no existen

Explicación campo por campo:

- **engine.name**: Identificador interno, aparece en los logs
- **db.***: Conexión a la base de datos de este nodo
- **group.id**: Rol del nodo (debe coincidir con **sym_node_group**)
- **external.id**: ID único, usado en enrutamiento y filtros
- **registration.url**: URL del nodo central para registrarse (vacío si ES el central)
- **sync.url**: URL donde ESTE nodo expone su servicio de sincronización
- **auto.registration**: Si está en **true**, intenta registrarse automáticamente
- **initial.load.create.first**: Crea triggers y estructuras automáticamente

Nodo Sucursal (Guayaquil)

Archivo: **guayaquil.properties**

properties

```
engine.name = guayaquil
db.driver = org.postgresql.Driver
db.url = jdbc:postgresql://postgres-guayaquil:5432/guayaquil_db
db.user = postgres
db.password = postgres

group.id = sucursal      # Pertenecce al grupo 'sucursal'
external.id = 002        # ID único (001, 002, 003, etc.)

# URLs - ¡IMPORTANTE!
registration.url = http://symmetricds:31415/sync/quito # Se conecta AL CENTRAL
sync.url =          # Vacío porque es un nodo hijo (no recibe conexiones)

auto.registration = true  # Se registra automáticamente al iniciar
initial.load.create.first = true
```

Diferencias clave con el central:

- `registration.url` apunta al central
- `sync.url` está vacío (no recibe conexiones de otros)
- `auto.registration = true`

Sincronización Unidireccional

Flujo: Central → Sucursales (solo en una dirección)

Caso de uso:

- Central mantiene catálogo de productos
- Sucursales solo leen, no modifican
- Actualizaciones solo ocurren en el central

Pasos Completos

1. Crear grupos de nodos (en BD Central)

```
sql
```

```
INSERT INTO sym_node_group (node_group_id, description)
VALUES ('central', 'Nodo Central - Matriz')
ON CONFLICT (node_group_id) DO NOTHING;
```

```
INSERT INTO sym_node_group (node_group_id, description)
VALUES ('sucursal', 'Nodos de Sucursales')
ON CONFLICT (node_group_id) DO NOTHING;
```

2. Crear enlace unidireccional (en BD Central)

```
sql

INSERT INTO sym_node_group_link (
    source_node_group_id,
    target_node_group_id,
    data_event_action
) VALUES (
    'central',
    'sucursal',
    'W'
)
ON CONFLICT (source_node_group_id, target_node_group_id)
DO UPDATE SET data_event_action = EXCLUDED.data_event_action;
```

Solo este enlace = unidireccional

3. Crear canal (en BD Central)

```
sql

INSERT INTO sym_channel (channel_id, processing_order, max_batch_size, enabled)
VALUES ('default', 1, 1000, 1)
ON CONFLICT (channel_id) DO UPDATE SET
    processing_order = EXCLUDED.processing_order,
    max_batch_size = EXCLUDED.max_batch_size,
    enabled = EXCLUDED.enabled;
```

4. Crear trigger (en BD Central)

```
sql
```

```
INSERT INTO sym_trigger (  
    trigger_id,  
    source_table_name,  
    channel_id,  
    sync_on_insert,  
    sync_on_update,  
    sync_on_delete,  
    last_update_time,  
    create_time  
) VALUES (  
    'trg_clientes',  
    'clientes',  
    'default',  
    1, 1, 1,  
    NOW(),  
    NOW()  
)  
ON CONFLICT (trigger_id) DO UPDATE SET  
    sync_on_insert = EXCLUDED.sync_on_insert,  
    sync_on_update = EXCLUDED.sync_on_update,  
    sync_on_delete = EXCLUDED.sync_on_delete,  
    last_update_time = EXCLUDED.last_update_time;
```

5. Crear router (en BD Central)

sql

```

INSERT INTO sym_router (
    router_id,
    source_node_group_id,
    target_node_group_id,
    router_type,
    create_time,
    last_update_time
) VALUES (
    'rt_central_to_sucursal',
    'central',
    'sucursal',
    'default',
    NOW(),
    NOW()
)
ON CONFLICT (router_id) DO UPDATE SET
    source_node_group_id = EXCLUDED.source_node_group_id,
    target_node_group_id = EXCLUDED.target_node_group_id,
    router_type = EXCLUDED.router_type,
    last_update_time = EXCLUDED.last_update_time;

```

6. Conectar trigger con router (en BD Central)

```

sql

INSERT INTO sym_trigger_router (
    trigger_id,
    router_id,
    initial_load_order,
    enabled,
    create_time,
    last_update_time
) VALUES (
    'trg_clientes',
    'rt_central_to_sucursal',
    100,
    1,
    NOW(),
    NOW()
)
ON CONFLICT (trigger_id, router_id) DO UPDATE SET
    initial_load_order = EXCLUDED.initial_load_order,
    enabled = EXCLUDED.enabled,
    last_update_time = EXCLUDED.last_update_time;

```


7. Registrar el nodo central (en BD Central)

sql

```
INSERT INTO sym_node (  
    node_id,  
    node_group_id,  
    external_id,  
    sync_enabled,  
    created_at_node_id  
) VALUES (  
    '000',  
    'central',  
    '000',  
    1,  
    '000'  
)  
ON CONFLICT (node_id) DO NOTHING;
```

8. Pre-registrar sucursales (en BD Central)

sql

-- Para cada sucursal que quieras autorizar:

-- Sucursal Guayaquil (002)

```
INSERT INTO sym_node (  
    node_id,  
    node_group_id,  
    external_id,  
    sync_enabled,  
    sync_url,  
    created_at_node_id  
) VALUES (  
    '002',  
    'sucursal',  
    '002',  
    1,  
    NULL,  
    '000'  
)  
ON CONFLICT (node_id) DO NOTHING;
```

```
INSERT INTO sym_node_security (  
    node_id,  
    node_password,  
    registration_enabled,  
    registration_time,  
    initial_load_enabled,  
    created_at_node_id  
) VALUES (  
    '002',  
    'guayaquil123',  
    1,  
    NULL,  
    1,  
    '000'  
)  
ON CONFLICT (node_id) DO NOTHING;
```

-- Sucursal Cuenca (003)

```
INSERT INTO sym_node (node_id, node_group_id, external_id, sync_enabled, sync_url, created_at_node_id)  
VALUES ('003', 'sucursal', '003', 1, NULL, '000')  
ON CONFLICT (node_id) DO NOTHING;
```

```
INSERT INTO sym_node_security (node_id, node_password, registration_enabled, registration_time, initial_load_enabled, c  
VALUES ('003', 'cuenca123', 1, NULL, 1, '000')  
ON CONFLICT (node_id) DO NOTHING;
```

9. Iniciar SymmetricDS en las sucursales

Las sucursales se registrarán automáticamente y recibirán:

- La configuración completa (triggers, routers, etc.)
 - Carga inicial de datos existentes
 - Sincronización en tiempo real de cambios futuros
-

Sincronización Bidireccional

Flujo: Central ↔ Sucursales (ambas direcciones)

Caso de uso:

- Clientes pueden crearse en central o sucursales
- Ventas se registran en las sucursales
- Todo se sincroniza en ambas direcciones

Pasos Adicionales (después de unidireccional)

1. Crear enlace inverso (en BD Central)

```
sql

INSERT INTO sym_node_group_link (
    source_node_group_id,
    target_node_group_id,
    data_event_action
) VALUES (
    'sucursal',    -- Ahora las sucursales ENVÍAN
    'central',    -- Al central
    'W'
)
ON CONFLICT (source_node_group_id, target_node_group_id)
DO UPDATE SET data_event_action = EXCLUDED.data_event_action;
```

2. Crear router inverso (en BD Central)

```
sql
```

```

INSERT INTO sym_router (
    router_id,
    source_node_group_id,
    target_node_group_id,
    router_type,
    create_time,
    last_update_time
) VALUES (
    'rt_sucursal_to_central',
    'sucursal',    -- Origen: sucursales
    'central',    -- Destino: central
    'default',
    NOW(),
    NOW()
)
ON CONFLICT (router_id) DO UPDATE SET
    source_node_group_id = EXCLUDED.source_node_group_id,
    target_node_group_id = EXCLUDED.target_node_group_id,
    last_update_time = EXCLUDED.last_update_time;

```

3. Conectar trigger con router inverso (en BD Central)

```

sql

INSERT INTO sym_trigger_router (
    trigger_id,
    router_id,
    initial_load_order,
    enabled,
    create_time,
    last_update_time
) VALUES (
    'trg_clientes',
    'rt_sucursal_to_central', -- Router inverso
    100,
    1,
    NOW(),
    NOW()
)
ON CONFLICT (trigger_id, router_id) DO UPDATE SET
    initial_load_order = EXCLUDED.initial_load_order,
    enabled = EXCLUDED.enabled,
    last_update_time = EXCLUDED.last_update_time;

```

⚠ Consideraciones de Bidireccional

Manejo de Conflictos

Cuando dos nodos modifican el mismo registro simultáneamente:

Estrategia por defecto: Last-Write-Wins (el último gana)

- El cambio con timestamp más reciente sobrescribe el anterior
- Puede perder datos si no se maneja correctamente

Otras estrategias:

1. **Manual:** Enviar conflictos a una tabla para revisión humana
2. **Priority:** Dar prioridad al central sobre sucursales
3. **Custom:** Script personalizado para resolver conflictos

Configurar resolución de conflictos:

```
sql
```

```

INSERT INTO sym_conflict (
    conflict_id,
    source_node_group_id,
    target_node_group_id,
    target_channel_id,
    target_catalog_name,
    target_schema_name,
    target_table_name,
    detect_type,
    resolve_type,
    ping_back,
    resolve_changes_only,
    resolve_row_only,
    detect_expression
) VALUES (
    'conflict_clientes',
    'sucursal',
    'central',
    'default',
    NULL,
    'public',
    'clientes',
    'USE_TIMESTAMP',    -- Detecta por timestamp
    'NEWER_WINS',       -- Gana el más nuevo
    1,                  -- Notifica al origen
    0,
    0,
    NULL
);

```

Evitar Loops Infinitos

SymmetricDS maneja esto automáticamente con:

- `sync_on_incoming_batch`: Por defecto en 0 (no re-sincroniza cambios que vienen de otros nodos)
- Tracking interno de origen de cambios

Verificación y Troubleshooting

Queries de Verificación

Ver estructura completa de configuración

```
sql
```

```
SELECT
```

```
    t.trigger_id,  
    t.source_table_name,  
    t.channel_id,  
    tr.router_id,  
    r.source_node_group_id,  
    r.target_node_group_id,  
    tr.enabled,  
    tr.initial_load_order
```

```
FROM sym_trigger t
```

```
JOIN sym_trigger_router tr ON t.trigger_id = tr.trigger_id
```

```
JOIN sym_router r ON tr.router_id = r.router_id
```

```
ORDER BY t.source_table_name, tr.initial_load_order;
```

Ver nodos registrados

```
sql
```

```
SELECT
```

```
    n.node_id,  
    n.node_group_id,  
    n.external_id,  
    n.sync_enabled,  
    ns.registration_enabled,  
    ns.registration_time,  
    ns.initial_load_enabled,  
    ns.initial_load_time
```

```
FROM sym_node n
```

```
LEFT JOIN sym_node_security ns ON n.node_id = ns.node_id
```

```
ORDER BY n.node_id;
```

Ver enlaces entre grupos

```
sql
```

```
SELECT
```

```
    source_node_group_id,  
    target_node_group_id,  
    data_event_action
```

```
FROM sym_node_group_link
```

```
ORDER BY source_node_group_id;
```

Ver estado de sincronización por nodo

```
sql
```

```
SELECT
    node_id,
    channel_id,
    queue_length,
    data_event_count,
    last_update_time
FROM sym_node_channel_ctl
WHERE node_id != '000' -- Excluir el nodo central
ORDER BY node_id, channel_id;
```

Ver cambios pendientes de sincronizar

```
sql

SELECT
    data_id,
    table_name,
    event_type,
    trigger_hist_id,
    row_data,
    create_time
FROM sym_data
WHERE data_id NOT IN (
    SELECT data_id FROM sym_data_event WHERE data_id IS NOT NULL
)
ORDER BY create_time DESC
LIMIT 20;
```

Ver batches de sincronización recientes

```
sql

SELECT
    batch_id,
    node_id,
    channel_id,
    status,
    data_event_count,
    byte_count,
    create_time,
    sent_count,
    load_count
FROM sym_outgoing_batch
ORDER BY batch_id DESC
LIMIT 20;
```


Errores Comunes

Error: "violates foreign key constraint sym_fk_rt_2_grp_lnk"

Causa: Intentas crear un router sin el `sym_node_group_link`

Solución: Crear el link primero

```
sql

INSERT INTO sym_node_group_link (source_node_group_id, target_node_group_id, data_event_action)
VALUES ('central', 'sucursal', 'W');
```

Error: "was not allowed to register. Registration is not open"

Causa: El nodo no tiene permisos de registro en `sym_node_security`

Solución: Autorizar el registro

```
sql

INSERT INTO sym_node_security (node_id, node_password, registration_enabled, initial_load_enabled, created_at_node_id)
VALUES ('002', 'password123', 1, 1, '000');
```

Error: "null value in column node_password"

Causa: El campo `node_password` no puede ser NULL

Solución: Proporcionar una contraseña

```
sql

INSERT INTO sym_node_security (node_id, node_password, ...)
VALUES ('002', 'guayaquil123', ...); -- NO usar NULL
```

Error: "Key (node_id)=(002) is not present in table sym_node"

Causa: Intentas crear `sym_node_security` antes de `sym_node`

Solución: Crear en orden correcto

```
sql

-- 1. Primero el nodo
INSERT INTO sym_node (...) VALUES ('002', ...);

-- 2. Luego la seguridad
INSERT INTO sym_node_security (...) VALUES ('002', ...);
```

Los datos no se sincronizan

Checklist de diagnóstico:

1. Verificar que los nodos estén registrados:

```
sql

SELECT node_id, sync_enabled FROM sym_node;
```

2. Verificar que los triggers estén instalados:

```
sql

SELECT trigger_name FROM information_schema.triggers
WHERE trigger_name LIKE 'sym_%';
```

3. Ver si hay batches con error:

```
sql

SELECT batch_id, node_id, status, error_flag
FROM sym_outgoing_batch
WHERE error_flag = 1;
```

4. Revisar logs de SymmetricDS para detalles del error
-

Casos de Uso Comunes

Caso 1: Catálogo Centralizado (Unidireccional)

Escenario:

- Central mantiene productos, precios, promociones
- Sucursales solo consultan, no modifican
- Actualizaciones instantáneas en todas las sucursales

Configuración:

```
sql
```

-- Productos

```
INSERT INTO sym_trigger (trigger_id, source_table_name, channel_id, sync_on_insert, sync_on_update, sync_on_delete, la
VALUES ('trg_productos', 'productos', 'default', 1, 1, 1, NOW(), NOW());
```

```
INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, enabled, create_time, last_update_time)
VALUES ('trg_productos', 'rt_central_to_sucursal', 10, 1, NOW(), NOW());
```

-- Precios

```
INSERT INTO sym_trigger (trigger_id, source_table_name, channel_id, sync_on_insert, sync_on_update, sync_on_delete, la
VALUES ('trg_precios', 'precios', 'default', 1, 1, 1, NOW(), NOW());
```

```
INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, enabled, create_time, last_update_time)
VALUES ('trg_precios', 'rt_central_to_sucursal', 20, 1, NOW(), NOW());
```

Caso 2: Ventas Locales con Consolidación (Bidireccional)

Escenario:

- Ventas se registran en cada sucursal
- Central consolida todas las ventas
- Productos se mantienen en central

Configuración:

sql

```
-- UNIDIRECCIONAL: Productos (Central → Sucursales)
```

```
INSERT INTO sym_trigger (trigger_id, source_table_name, channel_id, sync_on_insert, sync_on_update, sync_on_delete, la
VALUES ('trg_productos', 'productos', 'default', 1, 1, 1, NOW(), NOW());
```

```
INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, enabled, create_time, last_update_time)
VALUES ('trg_productos', 'rt_central_to_sucursal', 10, 1, NOW(), NOW());
```

```
-- BIDIRECCIONAL: Ventas (Sucursales ↔ Central)
```

```
INSERT INTO sym_trigger (trigger_id, source_table_name, channel_id, sync_on_insert, sync_on_update, sync_on_delete, la
VALUES ('trg_ventas', 'ventas', 'default', 1, 1, 0, NOW(), NOW()); -- No sincronizar DELETES
```

```
-- Central → Sucursales (para consolidación)
```

```
INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, enabled, create_time, last_update_time)
VALUES ('trg_ventas', 'rt_central_to_sucursal', 100, 1, NOW(), NOW());
```

```
-- Sucursales → Central (para reportes)
```

```
INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, enabled, create_time, last_update_time)
VALUES ('trg_ventas', 'rt_sucursal_to_central', 100, 1, NOW(), NOW());
```

Caso 3: Sincronización Selectiva por Sucursal

Escenario:

- Cada sucursal solo recibe SUS clientes
- Usa columna `sucursal_id` para filtrar

Estructura de tabla:

```
sql

CREATE TABLE clientes (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100),
  sucursal_id VARCHAR(10), -- '001', '002', '003', etc.
  email VARCHAR(100)
);
```

Configuración:

```
sql
```

-- Router con filtro por columna

```
INSERT INTO sym_router (  
    router_id,  
    source_node_group_id,  
    target_node_group_id,  
    router_type,  
    router_expression, -- ← FILTRO AQUÍ  
    create_time,  
    last_update_time  
) VALUES (  
    'rt_clientes_por_sucursal',  
    'central',  
    'sucursal',  
    'column',  
    'sucursal_id=:EXTERNAL_ID', -- Solo envía si coincide con external_id del nodo  
    NOW(),  
    NOW()  
);  
  
INSERT INTO sym_trigger (trigger_id, source_table_name, channel_id, sync_on_insert, sync_on_update, sync_on_delete, la  
VALUES ('trg_clientes', 'clientes', 'default', 1, 1, 1, NOW(), NOW());  
  
INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, enabled, create_time, last_update_time)  
VALUES ('trg_clientes', 'rt_clientes_por_sucursal', 100, 1, NOW(), NOW());
```

Resultado:

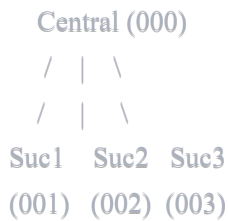
- Central inserta: `INSERT INTO clientes (nombre, sucursal_id) VALUES ('Juan', '002');`
- Solo la sucursal 002 (Guayaquil) recibe ese registro
- Sucursales 001, 003, etc. NO lo reciben

Caso 4: Hub and Spoke (Estrella)

Escenario:

- Un central con múltiples sucursales
- Sucursales NO se comunican entre sí
- Todo pasa por el central

Topología:



Configuración:

```
sql

-- Enlaces
INSERT INTO sym_node_group_link (source_node_group_id, target_node_group_id, data_event_action)
VALUES ('central', 'sucursal', 'W');

INSERT INTO sym_node_group_link (source_node_group_id, target_node_group_id, data_event_action)
VALUES ('sucursal', 'central', 'W');

-- NO crear enlaces entre sucursales
-- NO hacer: INSERT INTO sym_node_group_link VALUES ('sucursal', 'sucursal', 'W');
```

Scripts de Mantenimiento

Limpiar datos antiguos de sincronización

```
sql

-- Eliminar batches antiguos completados (más de 7 días)
DELETE FROM sym_outgoing_batch
WHERE status = 'OK'
AND create_time < NOW() - INTERVAL '7 days';

-- Eliminar eventos de datos procesados
DELETE FROM sym_data_event
WHERE batch_id IN (
  SELECT batch_id FROM sym_outgoing_batch WHERE status = 'OK'
);

-- Limpiar datos sincronizados
DELETE FROM sym_data
WHERE data_id NOT IN (SELECT data_id FROM sym_data_event);
```

Re-enviar carga inicial a un nodo

```
sql
```

-- Marcar para re-carga inicial

```
UPDATE sym_node_security
SET initial_load_enabled = 1,
    initial_load_time = NULL
WHERE node_id = '002';
```

-- SymmetricDS detectará esto y enviará toda la data nuevamente

Pausar sincronización temporalmente

sql

-- Deshabilitar un canal

```
UPDATE sym_channel
SET enabled = 0
WHERE channel_id = 'default';
```

-- Deshabilitar un trigger específico

```
UPDATE sym_trigger_router
SET enabled = 0
WHERE trigger_id = 'trg_clientes';
```

-- Para reanudar, poner enabled = 1

Monitorear performance

sql

-- Batches más lentos

SELECT

batch_id,

node_id,

channel_id,

data_event_count,

byte_count,

EXTRACT(EPOCH FROM (load_time - create_time)) as seconds_to_load

FROM sym_outgoing_batch

WHERE status = 'OK'

ORDER BY seconds_to_load DESC

LIMIT 20;

-- Tablas con más cambios

SELECT

table_name,

COUNT(*) as change_count

FROM sym_data

WHERE create_time > NOW() - INTERVAL '1 day'

GROUP BY table_name

ORDER BY change_count DESC;

Resumen: Orden de Ejecución Completo

Setup Unidireccional (Central → Sucursales)

En la BD del Central:

sql

-- 1. Grupos

```
INSERT INTO sym_node_group VALUES ('central', 'Nodo Central');
```

```
INSERT INTO sym_node_group VALUES ('sucursal', 'Sucursales');
```

-- 2. Link UNIdireccional

```
INSERT INTO sym_node_group_link VALUES ('central', 'sucursal', 'W');
```

-- 3. Canal

```
INSERT INTO sym_channel VALUES ('default', 1, 1000, 1);
```

-- 4. Trigger

```
INSERT INTO sym_trigger VALUES ('trg_clientes', 'clientes', 'default', 1, 1, 1, NOW(), NOW());
```

-- 5. Router

```
INSERT INTO sym_router VALUES ('rt_central_to_sucursal', 'central', 'sucursal', 'default', NOW(), NOW());
```

-- 6. Trigger-Router

```
INSERT INTO sym_trigger_router VALUES ('trg_clientes', 'rt_central_to_sucursal', 100, 1, NOW(), NOW());
```

-- 7. Nodo central

```
INSERT INTO sym_node VALUES ('000', 'central', '000', 1, '000');
```

-- 8. Autorizar sucursales

```
INSERT INTO sym_node VALUES ('002', 'sucursal', '002', 1, NULL, '000');
```

```
INSERT INTO sym_node_security VALUES ('002', 'guayaquil123', 1, NULL, 1, '000');
```

En las Sucursales:

- Solo arrancar SymmetricDS con el .properties correcto
- Auto-registro hace el resto

Upgrade a Bidireccional (Central ↔ Sucursales)

En la BD del Central (ejecutar DESPUÉS del unidireccional):

```
sql
```

-- 1. Link inverso

```
INSERT INTO sym_node_group_link VALUES ('sucursal', 'central', 'W');
```

-- 2. Router inverso

```
INSERT INTO sym_router VALUES ('rt_sucursal_to_central', 'sucursal', 'central', 'default', NOW(), NOW());
```

-- 3. Trigger-Router inverso

```
INSERT INTO sym_trigger_router VALUES ('trg_clientes', 'rt_sucursal_to_central', 100, 1, NOW(), NOW());
```

-- ¡Listo! Ahora es bidireccional

Glosario de Términos

- **Node Group:** Agrupación lógica de nodos con el mismo rol
- **Node Group Link:** Relación direccional entre grupos (quién puede enviar a quién)
- **Channel:** Agrupa tablas relacionadas, controla prioridad y tamaño de lotes
- **Trigger:** Detecta cambios en una tabla específica
- **Router:** Define a qué nodos destino van los cambios
- **Trigger Router:** Conecta un trigger con un router (activa la sincronización)
- **Node:** Instancia individual de base de datos en la red
- **Node Security:** Controla autenticación y permisos de registro
- **Batch:** Conjunto de cambios agrupados para sincronización
- **Initial Load:** Carga completa de datos históricos cuando un nodo se registra
- **Push:** El origen envía cambios activamente
- **Pull:** El destino solicita cambios periódicamente
- **Conflict:** Cuando dos nodos modifican el mismo registro simultáneamente

Referencias y Documentación Oficial

- **Documentación oficial:** <https://www.symmetricds.org/docs>
- **Guía de usuario:** <https://www.symmetricds.org/docs/user-guide/html/>
- **Referencia de tablas:** <https://www.symmetricds.org/docs/reference/html/>
- **Foro de la comunidad:** <https://sourceforge.net/p/symmetricds/discussion/>

Notas Finales

Buenas Prácticas

1. **Siempre usar transacciones** al configurar múltiples tablas relacionadas
2. **Probar en desarrollo** antes de producción
3. **Monitorear logs** regularmente para detectar problemas temprano
4. **Hacer backups** antes de cambios importantes en configuración
5. **Documentar** qué tablas se sincronizan y por qué
6. **Establecer política de conflictos** antes de ir a producción con bidireccional

Limitaciones Conocidas

- SymmetricDS sincroniza **datos**, no estructura de tablas (DDL)
- Cambios en estructura (ALTER TABLE) deben hacerse manualmente en todos los nodos
- Las columnas BLOB/CLOB grandes pueden afectar performance
- La sincronización en tiempo real depende de la red

Troubleshooting Tips

1. **Revisar logs primero** - la mayoría de errores están ahí
2. **Verificar conectividad** entre nodos
3. **Confirmar que los triggers existen** en la BD
4. **Comprobar permisos** de usuario de BD
5. **Validar configuración** con queries de verificación

Ejemplo Completo: Proyecto de E-Commerce

Escenario Real:

- Central en Quito con bodega principal
- Sucursales en Guayaquil, Cuenca, Ambato
- Catálogo centralizado, ventas locales

Estructura de Tablas

sql

-- Maestros (solo desde central)

```
CREATE TABLE productos (  
  id SERIAL PRIMARY KEY,  
  codigo VARCHAR(50) UNIQUE,  
  nombre VARCHAR(200),  
  precio DECIMAL(10,2),  
  activo BOOLEAN DEFAULT true  
);
```

```
CREATE TABLE categorias (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100),  
  descripcion TEXT  
);
```

-- Transaccionales (bidireccional)

```
CREATE TABLE ventas (  
  id SERIAL PRIMARY KEY,  
  sucursal_id VARCHAR(10),  
  fecha TIMESTAMP DEFAULT NOW(),  
  total DECIMAL(10,2),  
  cliente_id INTEGER  
);
```

```
CREATE TABLE ventas_detalle (  
  id SERIAL PRIMARY KEY,  
  venta_id INTEGER REFERENCES ventas(id),  
  producto_id INTEGER REFERENCES productos(id),  
  cantidad INTEGER,  
  precio_unitario DECIMAL(10,2)  
);
```

-- Local a cada sucursal (NO sincronizar)

```
CREATE TABLE inventario_local (  
  id SERIAL PRIMARY KEY,  
  producto_id INTEGER,  
  cantidad INTEGER,  
  ubicacion VARCHAR(50)  
);
```

Configuración Completa

sql

```
-- === EN BD CENTRAL (QUITO) ===
```

```
-- Grupos y Links
```

```
INSERT INTO sym_node_group VALUES ('central', 'Bodega Central');
INSERT INTO sym_node_group VALUES ('sucursal', 'Puntos de Venta');
INSERT INTO sym_node_group_link VALUES ('central', 'sucursal', 'W');
INSERT INTO sym_node_group_link VALUES ('sucursal', 'central', 'W');
```

```
-- Canales con prioridades
```

```
INSERT INTO sym_channel VALUES ('config', 1, 100, 1); -- Alta prioridad
INSERT INTO sym_channel VALUES ('transaction', 5, 1000, 1); -- Media
INSERT INTO sym_channel VALUES ('reference', 10, 5000, 1); -- Baja
```

```
-- UNIDIRECCIONAL: Productos (Central → Sucursales)
```

```
INSERT INTO sym_trigger VALUES ('trg_productos', 'productos', 'config', 1, 1, 1, NOW(), NOW());
INSERT INTO sym_router VALUES ('rt_productos_down', 'central', 'sucursal', 'default', NOW(), NOW());
INSERT INTO sym_trigger_router VALUES ('trg_productos', 'rt_productos_down', 10, 1, NOW(), NOW());
```

```
INSERT INTO sym_trigger VALUES ('trg_categorias', 'categorias', 'config', 1, 1, 1, NOW(), NOW());
INSERT INTO sym_trigger_router VALUES ('trg_categorias', 'rt_productos_down', 5, 1, NOW(), NOW());
```

```
-- BIDIRECCIONAL: Ventas (Central ↔ Sucursales)
```

```
INSERT INTO sym_trigger VALUES ('trg_ventas', 'ventas', 'transaction', 1, 1, 0, NOW(), NOW());
INSERT INTO sym_router VALUES ('rt_ventas_down', 'central', 'sucursal', 'default', NOW(), NOW());
INSERT INTO sym_router VALUES ('rt_ventas_up', 'sucursal', 'central', 'default', NOW(), NOW());
INSERT INTO sym_trigger_router VALUES ('trg_ventas', 'rt_ventas_down', 100, 1, NOW(), NOW());
INSERT INTO sym_trigger_router VALUES ('trg_ventas', 'rt_ventas_up', 100, 1, NOW(), NOW());
```

```
INSERT INTO sym_trigger VALUES ('trg_ventas_detalle', 'ventas_detalle', 'transaction', 1, 1, 0, NOW(), NOW());
INSERT INTO sym_trigger_router VALUES ('trg_ventas_detalle', 'rt_ventas_down', 110, 1, NOW(), NOW());
INSERT INTO sym_trigger_router VALUES ('trg_ventas_detalle', 'rt_ventas_up', 110, 1, NOW(), NOW());
```

```
-- Registrar nodos
```

```
INSERT INTO sym_node VALUES ('000', 'central', '000', 1, '000');
```

```
-- Sucursales
```

```
INSERT INTO sym_node VALUES ('001', 'sucursal', '001', 1, NULL, '000');
INSERT INTO sym_node_security VALUES ('001', 'guayaquil123', 1, NULL, 1, '000');
```

```
INSERT INTO sym_node VALUES ('002', 'sucursal', '002', 1, NULL, '000');
INSERT INTO sym_node_security VALUES ('002', 'cuenca123', 1, NULL, 1, '000');
```

```
INSERT INTO sym_node VALUES ('003', 'sucursal', '003', 1, NULL, '000');
INSERT INTO sym_node_security VALUES ('003', 'ambato123', 1, NULL, 1, '000');
```

```
-- NOTA: La tabla inventario_local NO tiene trigger,  
-- por lo tanto NO se sincroniza (cada sucursal mantiene su propio inventario)
```

¡Y eso es todo! Con este documento tienes una guía completa para implementar sincronización con SymmetricDS. 🌈

TIP FINAL: Guarda este documento y los scripts SQL en tu repositorio. Te ahorrarás horas cuando necesites configurar nuevos nodos o troubleshootear problemas.